Sprint 4 – Week 1 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace that connects students with short-term, skill-based professional opportunities. The platform currently supports the full project lifecycle from account creation through project completion.

The system architecture continues to follow a layered full-stack design:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

Primary user roles:

- Student
- Organization
- Administrator

1.2 Sprint 1 – 3 Recap

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including user authentication and registration (JWT), role-based access control (Student / Organization), student and organization profile management, project posting, and basic project browsing and search.

Sprint 2 introduced the Application Workflow Layer, enabling students to apply to projects, track application status, and allowing organizations to accept or reject applications (US-1 through US-9, FR-1 through FR-9).

Sprint 3 completed the core system lifecycle by implementing deliverable submission (FR-10), deliverable review (FR-11), project completion (FR-12), the feedback system (FR-13), admin moderation (FR-14), and system-wide logging middleware (FR-15). At the conclusion of Sprint 3, all foundational functional requirements had been satisfied and the platform supported a full end-to-end micro-internship lifecycle

1.3 Sprint 4 Roadmap Context

Sprint 4 focuses on transforming MicroMatch from a workflow-complete marketplace into an intelligent, engagement-driven platform. The sprint introduces four major feature areas that were identified during earlier backlog refinement:

Planned Sprint 4 enhancements:

- Notifications system – FR-16 / US-16
- Messaging system – FR-17 / US-17
- Dashboard Analytics – FR-18 / US-18
- Profile enhancements – FR-19 / US-19
- Smart Matching & Ranking Algorithm – SA-1

The addition of SA-1 introduces a significant algorithmic component required for the capstone milestone.

Week 1 of Sprint 4 specifically targets the Notifications feature (FR-16), which improves user awareness and responsiveness across all major lifecycle events on the platform.

1.4 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6

FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15
FR-16: The system shall notify users of key events (application decisions, deliverable reviews, project completion).	US-16
FR-17: The system shall allow users to send/receive messages within a project.	US-17
FR-18: The system shall provide users with basic analytics (applications, completed projects).	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19
SA-1: Smart Matching & Ranking Algorithm.	US-20

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 4 – Week 1 Overview

2.1 Sprint Goal

The overarching goal of Sprint 4 is:

- To transform MicroMatch from a workflow-complete marketplace into an intelligent engagement-driven platform through notifications, smart recommendations, messaging, analytics, and profile enhancements.

The goal of Sprint 4 – Week 1 is:

- ➢ Implement the Notifications System (FR-16) and finalize planning for the Smart Matching & Ranking Algorithm epic, ensuring Sprint 4 begins with strong user engagement foundations and technical risk reduction.

3. Sprint Planning

3.1 Product Backlog Grooming and Reprioritization

At the start of Sprint 4, the team conducted a backlog refinement session to assess remaining enhancements that were deferred from Sprint 3. All Sprint 3 requirements (FR-10 through FR-15) were confirmed as completed, and the Sprint 4 feature set (FR-16 through FR-19 plus the Significant Algorithm milestone SA-1) was reviewed and reprioritized.

Estimation was performed using Planning Poker with Fibonacci values (3, 5, 8, 13). Stories were evaluated based on:

- Dependency relationships between features
- Impact on user engagement and platform value
- Risk of integration failure
- Contribution to the capstone demonstration

Reprioritization Strategy

Notifications (FR-16) were designated as the Week 1 focus because they represent the lowest-friction, highest-impact improvement that can be layered on top of the existing workflow. Every major lifecycle event already exists in the backend (application decisions, deliverable reviews, project completion); the notification layer simply requires event hooks and a new data model, making it the ideal entry point for Sprint 4.

The Smart Matching Algorithm (SA-1) was scheduled for Week 2 as it is the most technically complex milestone in the sprint and benefits from notifications already being in place. Messaging, analytics, and profile enhancements were deferred to Week 3, as they are either dependent on or can be developed independently of the core Week 1–2 work.

3.2 Groomed Product Backlog

Priority Definitions

- Highest: Critical sprint milestone with significant algorithm or core infrastructure.
- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature

US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights
US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements
US-20 / SA-1	Smart Matching Algorithm	Highest	8	Sprint 4	Significant Algorithm

3.3 Sprint Backlog – Week 1 Commitment

Based on historical velocity and current capacity, the Week 1 Sprint 4 commitment is 11 story points across three GitHub issues that together deliver the complete notification feature end-to-end.

User Story	Description	Story Points
US-16	Notification Event API	5
US-16	Notification Retrieval API	3
US-16	Notification UI Center	3

Total Commitment: 11 Story Points

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity across previous sprints averaged:

- Sprint 1: ~16 story points
- Sprint 2: ~14 story points
- Sprint 3: ~16 story points

This establishes a consistent velocity range of 14–16 story points per week. The Week 1 commitment of 11 story points is intentionally slightly below average velocity to account for:

- New database model design (Notification entity + relationships)
- New API router with two endpoints
- Frontend component integration (navbar badge + notifications page)

- Testing and coverage requirements for a brand-new feature
- Sprint ramp-up overhead at the start of a new sprint phase

This conservative but structured commitment ensures the notification feature is delivered completely and correctly — with backend, API, and frontend all integrated — rather than leaving any layer incomplete entering Week 2.

3.4 Sprint 3 Backlog Structure

Sprint 4 was divided into three weekly development phases.

Week 1 – Notification Workflow

Completed Issues

Issue	Description	Story Points	User Stories	Status
#76	Notification Event API	5	US-16	Done
#83	Notification Retrieval API	3	US-16	Done
#84	Notification UI Center	3	US-16	Done

Week 1 Total Commitment: 11 Story Points

Week 2 – Smart Matching Algorithm

Planned Issues

Issue	Description	Story Points
#89	Matching Score Engine	8
#90	Recommendation API Endpoint	5
#91	Recommendation UI Integration	5

Week 2 Total Commitment: 18 Story Points

Week 3 — Messaging, Analytics, and Profile Enhancement Bundle

Planned Issues

Issue	Description	Story Points
#77	Project Messaging API	8
#85	Messaging UI Integration	5
#78	Analytics Summary API	5
#86	Analytics Dashboard UI	3

#79	Profile Enhancements API	3
#87	Profile UI Upgrade	3

Week 3 Total Commitment: 27 Story Points

4. Definition of Done (Sprint 4)

The Definition of Done was refined for Sprint 4 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model is designed and reviewed
- Database schema aligns with requirements
- API contract is clearly defined
- Endpoint request/response structures documented

Implementation

- Database migration successfully applied
- Backend endpoint implemented and fully functional
- Business logic correctly enforces requirements
- Role-Based Access Control (RBAC) enforced
- Input validation implemented (edge cases handled)
- Frontend components implemented (if applicable)
- Frontend successfully integrated with backend APIs

Testing

- Unit tests written for core logic
- Edge case scenarios covered
- API endpoints tested (success + failure cases)
- Manual functional testing completed
- No failing test cases

Quality & Integration

- Code coverage ≥ 90% fornew core business logic
- Code reviewed and approved

- Pull request merged into main branch
- No regression introduced to previous features
- System builds and runs without errors

Documentation

- API endpoints documented
- Code comments added where necessary
- User flow documented (if applicable)

Deployment Readiness

- Feature is deployable without breaking the existing system
- Environment variables/configurations updated as needed

5. Acceptance Criteria

5.1 Notification Event API

Requirement: FR-16 | **User Story:** US-16 | **Story Points:** 5

Acceptance Criteria:

- When an application is accepted or rejected, the corresponding student automatically receives a notification record in the database.
- When an organization reviews a deliverable (accepted or rejected), the submitting student receives an automatic notification.
- When a project is marked complete, all assigned students receive a project completion notification.
- All notifications default to an unread state upon creation (is_read = False).
- Each notification record persists to the database with: recipient user ID, message content, timestamp, and unread status.
- Notification creation does not break or alter the behavior of the existing workflow endpoints.
- Unauthorized or unauthenticated users cannot trigger or access notification creation logic directly.

Return Behavior:

Issue #76 implements event hooks that fire as side effects inside existing workflow endpoints (PUT /applications/{id}/status, PUT /deliverables/{id}/review, POST /projects/{id}/complete). There is no standalone HTTP response for notification creation — the parent endpoint's existing response is returned unchanged. The notification record created in the database must contain:

- id — auto-incremented primary key
- recipient_id — foreign key to the target user (student)
- message — human-readable description of the event (e.g., 'Your application was accepted')
- is_read — Boolean, defaults to False
- created_at — server-side timestamp of creation

5.2 Notification Retrieval API

Requirement: FR-16 | **User Story:** US-16 | **Story Points:** 3

Acceptance Criteria:

- GET /notifications returns only the notifications belonging to the currently authenticated user.
- Results are ordered from newest to oldest (descending by timestamp).
- A student cannot retrieve notifications intended for an organization user, and vice versa.
- PUT /notifications/{id}/read correctly marks the specified notification as read (is_read = True) in the database.
- Only the owning user may mark their own notifications as read; attempting to mark another user's notification returns HTTP 403.
- All endpoints require a valid JWT token; unauthenticated requests return HTTP 401.
- An empty list is returned (not an error) if the authenticated user has no notifications.

Return Behavior:

GET /notifications

Scenario	HTTP Status	Response Body
Authenticated user with notifications	200 OK	[{ "id": 1, "message": "...", "is_read": false, "created_at": "..." }, ...]

Authenticated user with no notifications	200 OK	[] (empty array — not a 404)
No or invalid JWT token	401 Unauthorized	{ "detail": "Not authenticated" }

PUT /notifications/{id}/read

Scenario	HTTP Status	Response Body
Notification marked read successfully200 OK	200 OK	[{ "id": 1, "message": "...", "is_read": false, "created_at": "..." }, ...]
Notification belongs to a different user	403 Forbidden	{ "detail": "Not authorized" }
Notification ID does not exist	404 Not Found	{ "detail": "Notification not found" }
No or invalid JWT token	401 Unauthorized	{ "detail": "Not authenticated" }

5.3 Notification UI Center

Requirement: FR-16 | **User Story:** US-16 | **Story Points:** 3

Acceptance Criteria:

- A bell icon is visible in the navigation bar for all authenticated users.
- An unread count badge is displayed on the bell icon when the user has at least one unread notification.
- Clicking the bell icon opens a dropdown preview showing the most recent notifications.
- A full notifications page is accessible, displaying all notifications for the user.
- Clicking on a notification in the UI triggers the mark-as-read API call and visually updates the read state.
- The unread badge count decrements correctly as notifications are marked read. The UI renders correctly on both desktop and mobile viewports (responsive layout).
- Notifications data is fetched from the backend API on page load; no hardcoded data is displayed.

Return Behavior:

Issue #84 is a frontend feature with no direct HTTP responses of its own. Observable state outcomes are the measure of correctness:

User Action	API Call Triggered	Expected UI Outcome
Page load / route change	GET /notifications → 200 OK	Bell badge displays unread count; no badge if count is 0

Click bell icon	None (local state toggle)	Dropdown renders with most recent notifications
Click unread notification	PUT /notifications/{id}/read → 200 OK	Item visually changes to read; badge count decrements by 1
Click unread notification (network error)	PUT /notifications/{id}/read → any error	Error state shown; no silent failure; badge unchanged
All notifications read	Badge count reaches 0	Bell icon displays with no badge

6. Source Code Development Progress

6.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

6.2 Summary of Contributions

Backend Contributions

- Designed and implemented the Notification ORM model with fields: id, recipient_id (FK → users), message, is_read, created_at.
- Established foreign key relationship between Notification and User (recipient) with cascade delete.
- Implemented automatic notification generation hooks by extending existing workflow endpoints:
    - Extended PUT /applications/{id}/status to emit notifications on accept/reject decisions.
    - Extended PUT /deliverables/{id}/review to emit notifications on deliverable review.
    - Extended POST /projects/{id}/complete to emit project completion notifications to all assigned students.
- All notifications default to is_read = False on creation.
- Notification persistence verified through DB session commit within the parent transaction.

- Implemented GET /notifications — returns all notifications for the authenticated user, ordered newest-first.
- Implemented PUT /notifications/{id}/read — marks a specific notification as read (is_read = True).
- RBAC enforcement: endpoints require valid JWT authentication; cross-user access returns 403.
- Pydantic schemas defined for NotificationRead response model.
- Empty result handled gracefully (returns empty list, not 404).

Frontend Contributions

- Added bell icon component to Layout.jsx navigation bar.
- Implemented unread count badge that dynamically reflects API data.
- Built dropdown preview component showing the N most recent notifications.
- Created dedicated /notifications page rendering all user notifications.
- Integrated click-to-read interaction: clicking a notification calls PUT /notifications/{id}/read and updates local state.
- Implemented responsive layout for both desktop and mobile viewports.
- Notification data fetched via Axios from the backend API on component mount

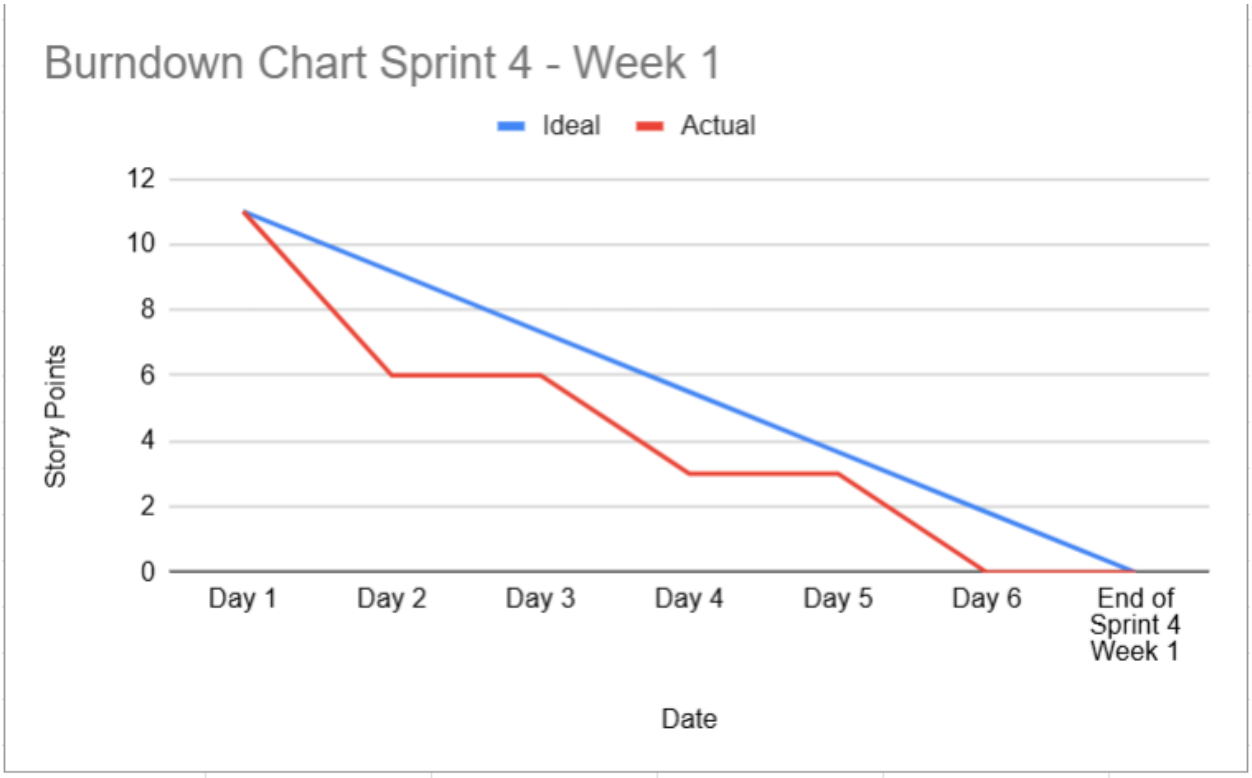
Integration Work

- Connected frontend notification components to backend API endpoints.
- Verified end-to-end flow: lifecycle event → notification creation → API retrieval → UI display.
- Ensured no regression to previously implemented workflow features.
- Validated RBAC behavior across notification endpoints.

6.3 Important Commits

Commit Message	Related Issue	User Story
feat: notification event api	#76	US-16
test: notification event api	#76	US-16
feat: notification retrieval api	#83	US-16
test: notification retrieval api	#83	US-16
ui: notification ui center	#84	US-16

6.4 Burndown Chart



Sprint Capacity: 11 Story Points
Sprint Duration: 6 Development Days

Burndown Chart Analysis

The burndown chart reflects steady, controlled progress across six development days, with backend work (Issues #76 and #83) occupying the first four days and frontend integration (Issue #84) completing the sprint in the final two days. This sequencing follows a deliberate backend-first strategy: the notification data model and API endpoints must exist before the UI can be meaningfully integrated.

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 average velocity: ~16 points
Sprint 4 Week 1 completed: 11 points

The slightly conservative commitment for Week 1 is intentional: it provides capacity buffer for the initial sprint setup overhead while still delivering a complete, fully integrated feature.

7. Software Testing

7.1 Requirement-to-Test Mapping

The following mapping demonstrates full traceability between functional requirements, user stories, and test cases for all Sprint 4 Week 1 backlog items.

Functional Requirement	User Story	Test Case ID
FR-16	US-16	TC-US16-01
FR-16	US-16	TC-US16-02
FR-16	US-16	TC-US16-03
FR-16	US-16	TC-US16-04
FR-16	US-16	TC-US16-05
FR-16	US-16	TC-US16-06
FR-16	US-16	TC-US16-07
FR-16	US-16	TC-US16-08
FR-16	US-16	TC-US16-09

7.2 User Acceptance Test Specification

All test cases are mapped to FR-16 ensuring full coverage of sprint scope.

TC-US16-01 – Notification Generated on Application Acceptance

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Organization

Preconditions:

- Organization account exists and is authenticated.
- Student has a pending application for the organization's project.
- Project exists and is in open status.

Steps:

1. Navigate to the login page and log in with valid organization credentials.
2. Navigate to the 'Applications' section for the relevant project.

3. Locate the student's pending application.
4. Click 'Accept Application'.
5. System processes the status change via PUT /applications/{id}/status.
6. Backend notification hook fires and creates a Notification record for the student.
7. Verify in the database that a notification record exists with the student's user ID, a relevant message, is_read = False, and a current timestamp.

Expected Result:

- **HTTP 200 OK returned for the application status update.**
- A Notification record is created in the database linked to the student's user ID.
- Notification message references the application acceptance event.
- is_read defaults to False.
- Timestamp is recorded correctly.
- The student's GET /notifications endpoint now returns this notification.

Postconditions:

- Notification persists in the database.
- Application status updated to 'accepted'.
- Student can view the notification via the API or UI.

TC-US16-02 – Notification Generated on Application Rejection

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Organization

Preconditions:

- Organization account exists and is authenticated.
- Student has a pending application for the organization's project.

Steps:

1. Log in with valid organization credentials.
2. Navigate to the relevant project's applications.
3. Locate the student's pending application.
4. Click 'Reject Application'.
5. System processes the status change via PUT /applications/{id}/status.
6. Backend notification hook fires.

Expected Result:

- HTTP 200 OK returned.
- A Notification record is created for the student indicating rejection.
- is_read = False on creation.
- Notification is accessible via GET /notifications for the student.

Postconditions:

- Application status updated to 'rejected'.
- Student can view the rejection notification.

TC- US16-03 – Notification Generated on Deliverable Review

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Organization

Preconditions:

- Student has submitted a deliverable for an accepted application.
- Organization user is authenticated.
- Deliverable status is 'submitted'.

Steps:

1. Log in as Organization.
2. Navigate to the deliverable dashboard for the project.
3. Select the submitted deliverable.
4. Click 'Accept Deliverable' (or 'Reject Deliverable').
5. System processes the review via PUT /deliverables/{id}/review.
6. Notification hook fires and creates a record for the student.

Expected Result:

- HTTP 200 OK returned for the deliverable review.
- A Notification record is created for the submitting student.
- Message indicates whether the deliverable was accepted or rejected.
- is_read = False.

Postconditions:

- Deliverable status updated.

- Student receives notification of review outcome.

TC-US16-04 – Notification Generated on Project Completion

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Organization

Preconditions:

- Project exists with at least one accepted/assigned student.
- Organization user is authenticated.
- Deliverable has been reviewed.

Steps:

1. Log in as Organization.
2. Navigate to the project management section.
3. Click 'Mark Project Complete'.
4. System processes the completion event.
5. Notification hook fires for all assigned students.

Expected Result:

- Project status updated to 'completed'.
- A Notification record is created for each assigned student.
- Message references project completion.
- All notifications default to is_read = False.

Postconditions:

- All assigned students receive a completion notification.
- Notifications persist in the database.

TC-US16-05 – Successful Notification Retrieval

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Student

Preconditions:

- Student account exists and is authenticated.

- At least one notification exists in the database for this student.

Steps:

1. Log in as Student.
2. Navigate to the Notifications section or call GET /notifications.
3. System fetches all notifications for the authenticated user.
4. Results are returned ordered newest-first.

Expected Result:

- HTTP 200 OK returned.
- Response contains a list of notification objects for the current user only.
- Each object includes: id, message, is_read, created_at.
- Results are ordered from newest to oldest.
- No notifications from other users are included in the response.

Postconditions:

- Student can view all their notifications.
- No cross-user data exposure occurs.

TC-US16-06 – Mark Notification as Read

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Student

Preconditions:

- Student is authenticated.
- Student has at least one unread notification.

Steps:

1. Log in as Student.
2. Navigate to the Notifications page or call GET /notifications.
3. Identify an unread notification.
4. Click on the notification (UI) or call PUT /notifications/{id}/read.
5. System updates is_read = True in the database.

Expected Result:

- HTTP 200 OK returned.

- is_read updated to True in the database.
- Subsequent GET /notifications reflects the updated read state.
- Unread count in the navbar badge decrements by 1.
- The notification is no longer visually highlighted as unread.

Postconditions:

- Notification is marked read.
- Unread badge count reflects the change.

TC-US16-07 – Unauthorized Access to Notifications Blocked

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Unauthenticated User

Preconditions:

- No valid JWT token is present.

Steps:

1. Attempt to call GET /notifications without an Authorization header.
2. Attempt to call PUT /notifications/{id}/read without an Authorization header.

Expected Result:

- HTTP 401 Unauthorized returned for GET /notifications.
- HTTP 401 Unauthorized returned for PUT /notifications/{id}/read.
- No notification data is exposed.
- No state change occurs in the database.

Postconditions:

- System remains secure; unauthenticated access is blocked.

TC-US16-08 – Unread Badge Count Displayed Correctly

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Student

Preconditions:

- Student is authenticated.
- Student has 3 unread notifications.

Steps:

1. Log in as Student.
2. Navigate to the home page or any protected route.
3. Observe the navigation bar.

Expected Result:

- Bell icon is visible in the navbar.
- Unread count badge displays '3'.
- Badge is visually distinct (e.g., colored circle over bell icon).

Postconditions:

- User is aware of pending unread notifications.

TC-US16-09 – Notifications Page Renders Correctly

Related Requirement: FR-16
User Story: US-16 – Notifications
Actor: Student

Preconditions:

- Student is authenticated.
- Student has both read and unread notifications.

Steps:

1. Log in as Student.
2. Navigate to the /notifications page.
3. Observe the full notifications list.
4. Click on an unread notification.

Expected Result:

- All notifications for the user are displayed in reverse-chronological order.
- Unread notifications are visually distinguishable from read ones.
- Clicking an unread notification marks it as read (badge count decrements).
- Layout is responsive and renders correctly on mobile viewports.
- No errors or broken UI elements are visible.

Postconditions:

- User can review all notifications and manage read state from the UI.
- All UI interactions correctly reflect backend state.

8. Backlog Grooming

Backlog updates this week were intentional and requirement-driven:

- FR-16 notification work was elevated to Week 1 to improve immediate user awareness for existing workflows.
- US-20 / SA-1 (Smart Matching & Ranking Algorithm) was formally added to the backlog as a Significant Algorithm milestone. This was identified during Sprint 3 as a high-value feature that warrants dedicated tracking. It is scheduled for Sprint 4 Week 2.
- Week 3 bundle (messaging/analytics/profile) retained but not expanded to avoid overcommitting sprint capacity.

No scope contraction occurred; reprioritization was sequencing-based, not requirement removal.

Rationale

The expansion of Sprint 4 user stories into individual issues reflects a deliberate move toward greater implementation-level granularity. As the team enters the final sprint, precise tracking of backend and frontend work is critical for maintaining velocity visibility and ensuring the sprint goal is achievable within the capstone timeline. Story point adjustments were driven by concrete effort analysis rather than scope changes — the features themselves are unchanged from prior backlog definitions.

9. Risk Management

Risk	Likelihood	Impact	Mitigation
Notification hooks break existing workflow endpoints	Low	High	Implement hooks as additive side-effects; run full regression test suite
Cross-user notification data exposure	Low	High	Strict RBAC enforcement + query filter by

			authenticated user ID
Frontend badge count out of sync with backend	Medium	Low	Fetch notification count on every page load; update state on mark-read action
DB schema change introduces migration risk	Low	Medium	Run migration in isolated environment first; no changes to existing tables
Week 2 algorithm complexity higher than estimated	Medium	Medium	Conservative Week 1 scope leaves buffer; SA-1 is highest priority Week 2 item

10. Sprint 4 Week 1 Outcome

Sprint 4 Week 1 is positioned to deliver the complete notification system for MicroMatch, covering all three implementation layers: event generation (Issue #76), retrieval API (Issue #83), and frontend UI (Issue #84). Together, these 11 story points represent a fully integrated feature that directly improves user awareness and platform responsiveness across the entire micro-internship lifecycle.

The notification system is deliberately structured as the first week of Sprint 4 because it adds tangible, visible value to every existing workflow without requiring significant changes to prior implementations. Students will immediately benefit from automatic status updates on their applications and deliverables, while organizations gain passive confirmation that their decisions are being communicated.

Upon successful completion of Week 1, the team will be positioned to begin the Smart Matching & Ranking Algorithm (SA-1) in Week 2, which represents the most technically challenging milestone of the sprint and the capstone's Significant Algorithm requirement.